codewiki 使用手册

把三端代码与旧文档 compound 成可被验证的 LLM Wiki

codewiki 由 Go CLI `kb` 和一组 Claude Code skills 组成。它把移动端 monorepo（Android Kotlin/Java、iOS Swift、HarmonyOS ArkTS，另支持 JS/TS）和遗留文档整理成适合 agent 使用的知识库。每次 merge 到 master 都会做一次增量更新。每个页面都通过“符号 ID + 内容 hash”锚定到代码，CI 中的 `kb verify` 负责证明这些知识与当前代码一致。

1. 项目目标
2. 架构设计：三层结构与 compound 流程
3. 核心能力：命令一览
4. 使用方法：安装、配置、CI、skills
5. Demo 操作流程与实测结果（含 axios 真实仓库案例）
6. 最佳实践与引用
7. 局限与路线图

版本 0.1 · 2026-10-20 · 模块 `github.com/luckylucky0001/codewiki` · Go 1.24 + tree-sitter（cgo）· 文中所有数字均来自本仓库 `demo/sample-output/` 中的实际运行结果

1 项目目标

移动团队的"知识"通常散落在三端代码、过时的 wiki 和口口相传里。每次让 LLM 从零读仓库，成本高，结果也不稳定。LLM 一次性生成的文档则会漂移：代码改了，文档没改，也没人知道哪一页已经过时。codewiki 想解决三个问题：

1. 知识复利：沿用 Karpathy 的 LLM Wiki 模式（raw sources / wiki pages / schema）。每次合并只重写受影响的页面，知识持续累积，不必每次从头生成。
2. 可证明一致：页面 frontmatter 的 `sources` 记录"符号 ID + 归一化内容 hash"。CI 中 `kb verify` 检查所有锚点与 HEAD 是否一致，不一致就不发布。
3. 三端对齐：用同一套 `route_id`、testID、API 描述 Android / iOS / 鸿蒙的同一个功能，并输出机器可读的 facts，供 e2e 测试（兄弟项目 e2epilot）、coding agent 和 CI 使用。

> 设计原则：确定性的事情交给 CLI（tree-sitter 抽取、diff、plan、verify），理解交给 LLM（写解释和语义核对），一致性靠硬校验（锚点 hash、写集、staging 验证后再原子发布）。

目标用户与场景

角色	场景	使用方式
Coding agent	改代码前了解一个文件影响哪些功能与测试	`kb context <file>`、`kb mcp` 工具
e2e 测试（e2epilot）	根据变更只重跑受影响路由；建立需求 → 流程的追踪	`routes/testids/apis/requirements.json`、`changed_route_ids`
Tech lead / 新人	理解三端实现差异与共享组件	`pages/features/*`、`parity.md`、`kb search`
CI	merge 后自动更新知识并设置门禁	`.github/workflows/compound.yml`
文档负责人	找出与代码冲突的 spec 和旧文档	`kb discover-docs` + `kb ingest` → `_conflicts.md`

交付物一览

17
kb 子命令（全部支持 --json）

7
tree-sitter 语言：Kotlin/Java/Swift/ArkTS/JS/TS/TSX

4
Agent Skills：compound / ingest / query / lint

45
Go 测试（单元 + 集成），vet / gofmt 干净

2 架构设计

2.1 三层结构（Karpathy LLM Wiki）



图 1　三层结构：facts 由 CLI 确定性生成，pages 由 LLM 在受限写集内书写，全部经过 verify 门禁

2.2 Compound 流程（每次 merge 到 master）

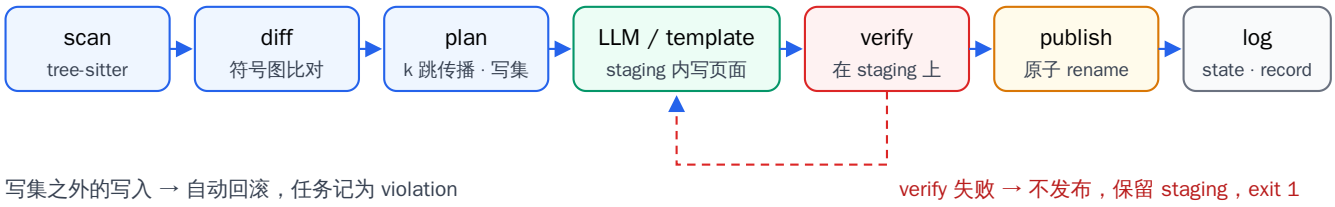


图 2　compound 流水线：基线是上次成功运行保存的 `.kb/graph.json`（不依赖 commit 链），因此 squash 合并或积压多次 merge 都能一次追平

2.3 符号模型与增量判定

字段	含义
`id`	`<platform>://<相对平台根路径>#<限定名>`，例如 `android://app/src/main/java/com/minishop/ui/cart/CartViewModel.kt#CartViewModel.addItem`
`hash`	去掉注释、归一化空白后 token 序列的 sha256。只改注释或格式视为 no-op。容器（class/struct）只计入自身头部和成员签名
`sig_hash`	声明头（含注解、装饰器）的 hash，用来区分 body 变化和 signature 变化
`refs`	函数体内引用的已知符号，按同文件 → 父类型被引用 → 全局唯一（结合调用点信息）的顺序解析
`norm_hash` / `simhash`	去掉自身名字后的 body 指纹，用于识别 rename（完全一致记 1.0，同目录 simhash ≥ 0.8 记模糊 rename）

plan 规则：页面被判为受影响，要么是它自己的锚点发生了 body / signature 变化、被删除或改名，要么它锚定了“签名变化符号”k 跳以内的调用方（沿反向引用边传播，参考 RepoAgent）。facts 中出现新 `route_id` 时创建功能页。模块变化超过 50%，或结构变化（增、删、改名、签名变化）超过 30% 时，改为全量重建。

锚点精度：功能页只锚定路由所属的 screen、它的 ViewModel / store，以及 2 跳以内的接口。被 3 个及以上路由引用的符号归入 `concepts/shared-components.md`，路由器和深链处理代码归入 `concepts/navigation.md`。这样修改 `BackButton` 时只会重新检查共享组件页（有单测覆盖）。

3 核心能力

所有命令都支持 `--json`：数据写 stdout，日志写 stderr。退出码：0 成功，1 存在 stale 或 verify 失败，2 用法错误，3 内部错误。会写文件的命令支持 `--dry-run`。所有命令都不交互，可以幂等地重复运行。

命令	作用
`kb init`	探测平台目录，生成 `codewiki.yaml`、wiki 骨架和 `AGENTS.md` schema，并把 `.kb-work/` 和扫描缓存写入 `.gitignore`
`kb scan`	用 tree-sitter 并行抽取符号和 facts，报告 `parse_failures`。按文件内容 hash 缓存解析结果
`kb diff [--base sha]`	按符号 ID 比较新旧图，分为 unchanged / body / signature / added / deleted / renamed，并输出 `changed_route_ids` 和 `changed_files`
`kb plan`	为每个受影响页面生成任务，带 reasons 和显式写集，写入 `.kb/plan.json`；超过阈值时改为全量
`kb compound`	scan → diff → plan → LLM → verify → 原子发布。支持 `--llm template|claude`、`--fallback`、`--max-llm-tasks`
`kb verify [--fix-index]`	CI 门禁：锚点 hash、符号是否存在、死链、孤儿页、index 覆盖、facts 和 parity 是否最新、requirements 锚点
`kb status`	列出 stale 页面及原因
`kb discover-docs`	按 spec-kit、OpenSpec、Kiro、ADR、RFC、AGENTS.md、README、docs/ 的约定发现文档并分类
`kb ingest`	把文档复制到 `raw/docs`，抽取断言（EARS、MUST/SHALL、FR-001、Kiro 编号），逐条判为 verified / conflict / unverifiable
`kb search / get / context`	检索 wiki；按 id 取页面；查询某个文件对应的页面、符号、路由、testID 和接口
`kb anchor`	打印符号的当前 hash，LLM 写 frontmatter 时使用
`kb lint-tid / parity`	testID 覆盖矩阵；功能 × 平台一致性矩阵和跨平台常量差异（生成 `pages/parity.md`）
`kb bench`	在合成仓库或真实仓库上对比冷 / 热扫描耗时
`kb mcp`	stdio MCP server，提供工具 `kb_search/get/context/anchor/status/parity`
`kb install-skills`	把 4 个 skill 安装到目标仓库的 `.claude/skills/`

三端 facts 来源

	路由 / 页面	深链	testID	接口
Android	Compose `composable(Routes.X)`，解析 `const val`	Manifest `<data scheme>`	`Modifier.testTag`	Retrofit `@GET/@POST`
iOS	`enum Route: String` + `XxxView`	Info.plist `CFBundleURLSchemes`	`.accessibilityIdentifier`	`send(.post, path:)`
鸿蒙	`router_map.json`（name / pageSourceFile / buildFunction）、`main_pages.json`	`module.json5` skills.uris	`.id('x')`（含 ArkUI 链式调用）	`http.createHttp().request`

动态 testID（如 `product_item_${id}`）按字面前缀 `product_item_` 记录。字符串拼接、插值和常量引用都会静态求值。输出的 JSON 与 e2epilot 共用同一份 schema（`{"version":1,"routes":[...]}`）。

4 使用方法

4.1 安装

```
go install github.com/luckylucky0001/codewiki/cmd/kb@latest   # 需要 Go 1.24 与 C 编译器（cgo）
# 或：git clone … && go build -o kb ./cmd/kb
# ArkTS 语法以源码形式 vendored 在 internal/grammars/arkts（MIT），go.mod 中没有 replace
```

4.2 初始化与配置

```
cd your-monorepo
kb init
kb discover-docs --json > /tmp/docs.json
kb ingest --manifest /tmp/docs.json \
   --kinds spec,design,adr,agent-guide,doc
kb compound            # 首次全量（template）
kb compound --llm claude
kb verify
```

```
# codewiki.yaml
version: 1
wiki_dir: wiki
platforms:
  - {name: android, root: android}
  - {name: ios,     root: ios}
  - {name: harmony, root: harmony}
llm:
  cmd: claude -p --permission-mode acceptEdits
...
plan:
  hops: 1
  rebuild_module_ratio: 0.5
  rebuild_structural_ratio: 0.3
```

可以用 `--config <file> --root <repo>` 把 wiki 写到被分析仓库之外，只读分析第三方仓库，axios 案例就是这样做的。

4.3 页面 frontmatter 契约

```
---
id: features/cart                       # = 路径去掉 pages/ 与 .md（verify 强制）
title: 购物车（cart）
kind: feature                           # feature | concept | overview | conflicts | parity
platforms: [android, harmony, ios]
routes: [cart]
sources:
  - sym: android://app/src/main/java/com/minishop/ui/cart/CartScreen.kt#CartScreen
    hash: 298d51bd3abcc95697e472891f857dc8…
source_commit: 516a1c3
status: current                         # current | stale | outdated-doc
---
```

4.4 CI 接入

`.github/workflows/compound.yml` 在 push 到 master/main 时触发。`concurrency` 保证串行，且不取消排队中的运行。`actions/cache` 缓存 kb 二进制和 `wiki/.kb/cache`（扫描缓存）。配置了 `ANTHROPIC_API_KEY` 时用 claude，否则用 template。`kb verify` 失败时 job 失败；成功时把 wiki 变更以 `[skip ci]` 提交回仓库。`examples/ci/README.md` 还说明了独立 wiki 仓库和 `kb` 孤立分支两种接法。

4.5 Skills 与 MCP

skill	何时使用
`kb-compound`	合并后增量更新；也是 `--llm claude` 时单个任务的写作规范（写集、frontmatter、链接规则）
`kb-ingest`	发现并核对 spec / 旧文档，产出 `_conflicts.md` 和 `requirements.json`
`kb-query`	回答“X 在三端怎么实现”，改代码前先跑 `kb context`
`kb-lint`	`kb verify`、`--fix-index`、`lint-tid`，以及错误码说明

本仓库的 `.claude/skills/*` 是指向 `skills/*` 的符号链接，Claude Code 可以直接发现；其他仓库用 `kb install-skills <repo>` 安装。也可以执行 `claude mcp add codewiki -- kb mcp --root <repo>`，把 wiki 作为 MCP 工具接入。

4.6 与 e2epilot 联动

```
ROUTES=$(kb diff --json | jq -r '.changed_route_ids | join(",")')   # 例：cart,product_detail
e2e graph invalidate --changed-routes "$ROUTES"
e2e impact --changed-routes "$ROUTES" --requirements wiki/facts/requirements.json
```

5 Demo 操作流程与实测结果

`demo/run.sh` 用 `examples/minishop` 新建一个临时 git 仓库，同一个电商 App 在 Android / iOS / 鸿蒙三端各实现一份：6 条路由，11 类 testID，5 个接口。脚本依次执行 init → discover-docs → ingest → 首次 compound，再把 3 个变更集以 `--no-ff` 合并到 master，每次合并后运行 status / diff / plan / compound / verify / parity。`DEMO_LLM=template` 约 1 分钟完成，结果可复现；`DEMO_LLM=claude` 使用真实的 `claude -p`。

```
DEMO_LLM=template demo/run.sh                          # 输出到 demo/output/<ts>-template/SUMMARY.md
DEMO_LLM=claude DEMO_MAX_LLM_TASKS=2 demo/run.sh    # 真实 LLM，单个任务约 30–60 s
```

5.1 文档发现与断言核对

共发现 8 份文档：spec-kit 的 `specs/001-cart/spec.md` 和 `plan.md`、Kiro 的 `.kiro/specs/login/requirements.md` 和 steering、ADR、两份旧文档。从中抽出 28 条断言。template 模式的结果是 verified 20 / conflict 6 / unverifiable 2。

断言（来源）	结论与证据（代码为准）
登录使用手机号 + 短信验证码，`POST /api/sms/send`（legacy/login.md）	conflict：代码中不存在该接口，只有 **`POST /api/login`**
购物车最多 10 件（`MAX_CART_ITEMS`）（legacy/cart.md）	conflict：android 与 harmony 为 50，ios 为 99，与文档的 10 都不一致
token 由 `SessionManager.saveToken()` 持久化（legacy/login.md）	conflict：类型 **`SessionManager`** 不存在
FR-002：购物车上限 99（spec-kit）	变更集 01 合并后由 verified 变为 conflict
FR-004：优惠券 `POST /api/coupons/apply`	conflict：功能未实现
LOGIN-R1.1：提交后调用 `POST /api/login` 并进入 `products`（Kiro）	verified：三端接口与路由都有锚点
LOGIN-R2.2：用户名为空时弹出短信验证对话框（Kiro）	template 判为 verified（testID 存在）；claude 判为 conflict：“LoginViewModel.login 仅设置 invalid_credentials，没有短信对话框”

5.2 三次合并的增量结果（template 模式实测）

步骤	diff	受影响路由	任务	verify
首次 compound	无基线，全量重建	—	10（6 个功能页 + 总览 + 2 个 concept + _conflicts）	PASS
01 购物车上限 99→50（iOS 漏改）+ 只改注释	body 2，unchanged 212；注释改动为 no-op	`cart`, `product_detail`	3：cart、product_detail、_conflicts	PASS
02 ArkTS 改名与签名变化	renamed 1（`Api.loadProducts → fetchProducts`，相似度 1.00），signature 1，body 4	`product_detail`, `products`	4	PASS
03 三端新增 profile 路由	added 10，body 4	`profile`（route_added）	4：create profile、overview、navigation、shared	PASS

最终 wiki 共 24 个页面、355 个锚点，verify 通过，`lint-tid` 无缺口。`kb parity` 发现了变更集 01 故意留下的三端不一致：`max_cart_items` 在 android、ios、harmony 分别是 `50 / 99 / 50`。

```
$ kb status        # 合并 01 之后、compound 之前
stale pages/_conflicts.md (2 anchors: CartViewModel.MAX_CART_ITEMS, MAX_CART_ITEMS)
stale pages/features/cart.md (1 anchors: CartViewModel.MAX_CART_ITEMS)
stale pages/features/product_detail.md (1 anchors: CartViewModel.MAX_CART_ITEMS)
stale pages/parity.md (generated facts out of date)
$ kb context android/app/src/main/java/com/minishop/ui/cart/CartViewModel.kt
pages (4):
  pages/_conflicts.md — 文档核对（冲突与已验证断言） [current] anchors=2
  pages/concepts/shared-components.md — 共享组件 [current] anchors=1
  pages/features/cart.md — 购物车（cart） [current] anchors=5
  pages/features/product_detail.md — 商品详情（product_detail） [current] anchors=5
```

5.3 claude 模式

claude 模式用 `claude -p --model sonnet`，每次 compound 最多 2 个 LLM 任务，其余用 template，首次全量也用 template，以控制成本。所有步骤 verify 均通过，最终得到 24 个页面、356 个锚点。断言核对结果为 verified 18 / conflict 8 / unverifiable 2，比 template 多出 2 条语义冲突：一条是上面的 LOGIN-R2.2；另一条是 spec-kit `plan.md` 写的“三端共享上限常量”，LLM 给出的结论是“上限常量各自定义且值不同（Android 50、Harmony 50、iOS 99），并非共享”。LLM 写出的页面带有三端对照表、失败处理差异和测试要点。例如结算页指出“iOS 的 `placeOrder` 用 `try?` 吞掉错误，失败时不跳转也没有提示”。购物车页发现了示例代码中的一个真实问题：`preload` 写入的 `pending` 列表从未被消费。CLI 会自动回滚写集之外的写入；集成测试用一个伪造的 LLM 写 `index.md`，验证这次写入被回滚。

5.4 真实仓库：axios（只读）

指标	结果
扫描	`lib/**/*.js` 共 70 个文件、323 个符号，0 个 parse failure；全仓 243 个文件、1009 个符号
文档发现	149 份（doc 133 / readme 7 / design 6 / agent-guide 3）
template 核对（16 份，174 条）	verified 89 / unverifiable 85 / conflict 0（误报已从 38 降到 0）
AGENTS.md：“default adapter preference is `['xhr','http','fetch']`”	verified，锚点 **`web://lib/defaults/index.js#defaults`**
claude 核对 AGENTS.md（48 条，1 次调用）	verified 23 / unverifiable 22 / conflict 3：规范要求 “never raw `Error`”，但 `toFormData.js` 抛出 `new Error('Circular reference detected')`；规范要求不直接使用 `bind`，但 `Axios.js` 中有 `dispatchRequest.bind(this)`

5.5 扫描性能（**`kb bench`**，4 核，实测）

仓库	文件 / 符号	冷扫描	热扫描（缓存全命中）	1% 文件变化
合成仓库（Kotlin / Swift / ArkTS 各 1000 个文件）	3000 / 21000	3711 ms	463 ms	974 ms
axios lib	70 / 323	191 ms	25 ms	—
axios 全仓	243 / 1009	993 ms	69 ms	—
examples/minishop	42 / 214	91 ms	8 ms	—

6 最佳实践与引用

实践	做法	参考
事实与叙述分离	CLI 抽取所有能确定的事实，并标记 DO NOT EDIT；LLM 只写解释。分 raw / wiki / schema 三层，index.md 作目录，log.md 只追加	Karpathy《LLM Wiki》
按符号增量	以符号 ID 和归一化 hash 判定变化，注释与格式改动不触发重写	FSoft CodeWiki 的增量设计
引用失效	签名变化沿反向引用边传播 k 跳	RepoAgent
受限写集	每个任务只能写自己的页面，越界写入自动回滚	OpenDeepWiki
repo-map 式上下文	给 LLM 的是精选的符号签名和源码片段；`facts/<p>/symbols.md` 就是一份 repo map	Aider repo-map
需求可追踪	用 EARS 句式（WHEN … THE SYSTEM SHALL …）或 FR-xxx 编号写需求，断言 ID 保持稳定，以代码为准	GitHub spec-kit、OpenSpec、Kiro、EARS
skill 保持薄	skill 只描述何时用、怎么调 CLI；SKILL.md 少于 500 行，详细内容按需放到 references/	Agent Skills 规范
先验证再发布	在 staging 上 verify，通过后再原子 rename；CI 串行执行，失败不发布	—

常见坑

- 手工修改 wiki 里的 hash 让 verify 变绿，会破坏“知识与代码一致”的保证。应该运行 `kb compound`。
- wiki 自身的提交又触发 compound，形成循环。用 `paths-ignore: wiki/**` 加 `[skip ci]` 避免。
- 路由在运行时拼接时无法静态求值。路由表应保持为常量或资源文件：`object Routes`、`enum Route: String`、`router_map.json`。
- 多个页面锚定同一个共享符号会导致大面积 stale。交给 shared-components / navigation 概念页处理即可。
- 修改抽取器后要提升 `scan.CacheVersion`，否则会读到旧缓存。

7 局限与路线图

当前局限

- cgo：tree-sitter 语法是 C 代码，构建需要 C 编译器，交叉编译需要对应的工具链。Swift 语法编译时会输出无害的 `TOKEN_COUNT redefined` 警告，可用 `CGO_CFLAGS=-w` 屏蔽。
- ArkTS 语法：使用 harmony-contrib/tree-sitter-arkts。上游报告约 94% 的真实工程文件能无错解析。出错的文件会记入 `parse_failures`，并尽量继续抽取符号，扫描不会中断。
- 引用解析：只做名称级解析，没有类型推断，也不建立跨平台的边。同名成员较多时可能偏保守，偶尔会误连（例如 Compose 的 `items(...)`）。
- template 写作器：生成的页面偏结构化索引，叙述质量需要 claude 模式。claude 模式单个任务约 30–60 秒，可用 `--max-llm-tasks` 控制成本。
- 断言核对：template 模式只覆盖存在性、字面量和数值，纯散文断言会被忽略；行为类断言需要 LLM 做语义核对。
- facts 启发式：iOS 的 API 抽取依赖 `send(.post, path:)` 这类封装；路由的 `route_id` 依赖常量名、枚举名或 router_map 的 name。

路线图

阶段	内容
短期	引入 SCIP / LSP 符号作为可选的精确引用来源；常量取值差异接入 CI 门禁（`kb parity --strict`）；为 template 写作器增加流程页（flows/）
中期	与 e2epilot 双向打通：按 `changed_route_ids` 只重跑受影响流程，并把测试结果回写到 requirements；按模块分片并行执行 LLM 任务
长期	多仓库 / 多 wiki 联邦检索；从 PR 评论增量吸收人工知识（经 verify 后才能沉淀）；更多语言（Dart / C++）

> 仓库结构：`cmd/kb` 入口 · `internal/extract` tree-sitter 抽取 · `internal/scan` 并行扫描与缓存 · `internal/facts` 三端事实 · `internal/engine` diff / plan / compound / claims / parity / impact · `internal/wiki` 页面与 verify · `internal/cli` 命令与 MCP · `internal/grammars/arkts` vendored 语法 · `skills/` · `examples/` · `demo/`